CS 528 · CLOUD COMPUTING

Homework 6

Ruoxi Cao Fall 2026 · November 4, 2026

1 Overview

This homework converts the database schema in HW5 to one that conforms to the 3rd normal form and transfers data from the old table to the new one. An App is created to train 2 models to predict some of the fields using some other fields.

- Github repo: https://github.com/ruoxicao77i/cs528-hw6
- GCP project: `tonal-plateau-508118-g0`, region `us-central1`

2 How to Configure and Run

File	Purpose
`sql/migrate_3nf.sql`	Creates new schemas and transfers data, executed once
`src/db.py`	Connects to Cloud SQL, executes sql queries and provides data to the models
`src/country_model.py`	Model 1: `IP` → `country`
`src/income_model.py`	Model 2: `other fields (age essentially)` → `income`
`src/main.py`	entry: fetch data, train 2 models and upload the results to the bucket
`requirements.txt`	
`scripts/setup.sh`	
`scripts/run_on_vm.sh`	
`scripts/cleanup.sh`	

2.1 Prerequisites

- **Access to the project.** The TA account `jskyi@bu.edu` has been granted access to the project `tonal-plateau-508118-g0` (Editor, Project IAM Admin and Storage Admin). If you use another Google account, please let me know and I will grant it the same access.
- `gcloud` installed and logged in with access to the project
- Python 3
- `Cloud SQL` instance `hw5-db` is RUNNABLE and prepopulated
- Cloud Storage bucket `cs528-ruoxi-pagerank` from the previous homework; the outputs are written under `hw6/`

2.2 Setup

```
gcloud auth login jskyi@bu.edu

git clonehttps://github.com/ruoxicao77i/cs528-hw6 hw6
cd hw6

bash scripts/setup.sh
```

The `setup.sh` handles following issues:

- enabling apis
- activate the database
- create the VM and authorize it
- run the program in the VM;
- call `cleanup.sh`, which deletes the VM

3 Schema Normalization

3.1 Why the Old Schema Is Not in 3NF

The original schema

`requests(id, country, client_ip, gender, age, income,is_banned, time_of_day, requested_file)`

does not conform to the third normal form. A table is in 3NF when every non-key attribute depends directly on the primary key, and not transitively through another non-key attribute. The old table has two such transitive dependencies:

- `country` → `is_banned`: whether a request is banned is completely determined by its country, so `is_banned` depends on the key `id` only through `country`.
- `client_ip` → `country`: as the assignment hint states, a given IP address always comes from the same country, so `country` depends on `id` only through `client_ip`.

Together, they form the chain

$$\texttt{id} \rightarrow \texttt{client_ip} \rightarrow \texttt{country} \rightarrow \texttt{is_banned}$$

As a result, the country of each IP and the banned status of each country are stored repeatedly in every request row, which wastes space and allows inconsistent values.

3.2 New Schema

To remove the transitive dependencies, we move each dependency into its own table. The attribute on the left side of the dependency becomes the primary key of the new table:

- `country` → `is_banned` becomes the table `countries`;

- `client_ip` → `country` becomes the table `ip_addresses`;
- The remaining attributes describe one request, so they constitute the table `requests_3nf`.

```sql
CREATE TABLE countries (
    country_id INT AUTO_INCREMENT PRIMARY KEY,
    country_name VARCHAR(100) NOT NULL UNIQUE,
    is_banned BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE ip_addresses (
    client_ip  VARCHAR(45) PRIMARY KEY,
    country_id INT NOT NULL,
    FOREIGN KEY (country_id) REFERENCES countries(country_id)
);

CREATE TABLE requests_3nf (
    id  INT AUTO_INCREMENT PRIMARY KEY,
    client_ip   VARCHAR(45) NOT NULL,
    gender      VARCHAR(16),
    age         VARCHAR(16),
    income      VARCHAR(32),
    time_of_day DATETIME,
    requested_file  VARCHAR(255),
    FOREIGN KEY (client_ip) REFERENCES ip_addresses(client_ip)
);
```

Listing 1: New schema in 3NF

Each table now only has direct dependencies on its key:

Table	Dependencies
`countries`	`country_id` → `country_name`, `is_banned`
`ip_addresses`	`client_ip` → `country_id`
`requests_3nf`	`id` → all other fields

Table 1: Functional dependencies in the new schema

Some design choices are worth noting:

- `country_name` is `UNIQUE`, so each country is stored only once. This makes `country_name` a candidate key, so `countries` is still in 3NF.
- `country_id` is a surrogate key. An integer key is shorter and faster to join than a country name. It is a design choice, not a requirement of 3NF.
- `gender`, `age` and `income` stay in `requests_3nf`. One IP can be shared by many people(for example, through NAT or a proxy), so these fields describe the sender of one request, not the IP address.

3.3 Data Migration

The migration script `sql/migrate_3nf.sql` creates the new tables and copies the data from the old table. It runs in four steps:

1. Check if the data are legal. Before data migration, we need to check the consistency: each ip has only one country, each each country has only one banned status.
2. Fill the `countries`.
3. Fill the `ip_addresses`.
4. Fill `requests_3nf`. Copy each request and keep its original `id`, so the rows in the old and new tables can be matched one to one.

```sql
-- 0. Check if the data are legal (The results should be null)
SELECT client_ip FROM requests
GROUP BY client_ip HAVING COUNT(DISTINCT country) > 1;

SELECT country FROM requests
GROUP BY country HAVING COUNT(DISTINCT is_banned) > 1;

-- 1. countries
INSERT INTO countries (country_name, is_banned)
SELECT country, MAX(is_banned)
FROM requests
WHERE country is NOT NULL
GROUP BY country;

-- 2. ip_addresses
INSERT INTO ip_addresses (client_ip, country_id)
SELECT r.client_ip, MIN(c.country_id)
FROM requests r
JOIN countries c ON c.country_name = r.country
WHERE r.client_ip IS NOT NULL
GROUP BY r.client_ip;

-- 3. requests table in 3nf
INSERT INTO requests_3nf
    (id, client_ip, gender, age, income, time_of_day, requested_file)
SELECT r.id, r.client_ip, r.gender, r.age, r.income, r.time_of_day, r.requested_file
FROM requests r
JOIN ip_addresses i ON r.client_ip = i.client_ip;
```

Listing 2: Data Migration

After the migration, the old table has 100034 rows and `requests_3nf` has 99999 rows, some requests are dropped because some requests do not have `country` attribute when we sent some test requests without header information in HW5.

4 Models

4.1 Model1: IP to country

Goal. The first model predicts the country of a request from its client IP. As shown in Section 3, `client_ip` determines `country`, so this is a well defined classification problem.

Data. We join `ip_addresses` with `countries` to get one row per IP address with its country. Since `client_ip` is the primary key of `ip_addresses`, each IP appears only once.

Features. A model cannot use an IP string directly, so we convert each IPv4 address `a.b.c.d` into a 32-bit integer:

$$a \times 256^3 + b \times 256^2 + c \times 256 + d$$

Train/test split. The data is split into 80% training data and 20% test data, with random seed 42 by default. Because each IP appears only once, no IP in the test set is seen during training. So the model cannot simply memorize the answer. It has to learn a rule that works for new IPs.

Model. We use `DecisionTreeClassifier` from scikit-learn. In practice, IP addresses are assigned to countries in blocks, so each country owns one or more ranges of integers. A decision tree splits the data with rules like "IP $\leq$ some value", which cuts the number line into ranges. This matches how IPs are assigned, so a tree can learn the country ranges with only a few splits. The code is in `src/country_model.py`.

4.2 Model 2: Fields to Income

Goal. The second model predicts the income of the sender from the other fields of a request. Income is stored as one of eight ranges, such as `20k-40k`, so we treat it as a classification problem with eight classes.

Data. We join the three tables to get one row per request, and drop rows with missing values. This gives 99999 rows. The eight income levels are almost equally common (about 12.5% each), so always guessing the most common level gives a baseline accuracy of only 12.7%.

First attempt. We first used all available fields as features: `gender`, `age`, `country`, `is_banned`, and the hour extracted from `time_of_day`. Text fields were one-hot encoded, and we trained a random forest. With seed 42 it reached 34.1% on the test set, but its accuracy on the training set was 76.83%. This large gap means the model was memorizing noise in the training data rather than learning a real pattern. To find out which fields actually matter, we looked at the data directly.

Feature analysis. For each field, we compared the income distribution inside each group with the overall distribution.

- `gender`, `is_banned` and the hour of the request show no relation to income. In every group, each income level still has a share of about 12.5%.
- Combining `age` with `gender` does not help either. Inside each age group, men and women have the same income distribution.
- `country` has 197 values and `requested_file` has almost 12,000, which is too many to inspect by hand. Many of their groups are also very small, so their tables are dominated by noise.

We tested `country` with a decision tree instead (Table 3): adding it to `age` lowers the test accuracy from 33.6% to 33.4%. `requested_file` is almost unique for each request, so it cannot generalize to new data.

- `age` is the only useful field, and its pattern is very regular. Each age group contains exactly three income levels, each with a share of about one third (Table 2).

Age	Possible incomes (about 1/3 each)
0–16	0-10k, 10k-20k, 250k+
17–25	0-10k, 10k-20k, 20k-40k
26–35	10k-20k, 20k-40k, 40k-60k
36–45	20k-40k, 40k-60k, 60k-100k
46–55	40k-60k, 60k-100k, 100k-150k
56–65	60k-100k, 100k-150k, 150k-250k
66–75	100k-150k, 150k-250k, 250k+
76+	0-10k, 150k-250k, 250k+

Table 2: Income levels that appear in each age group

Final model. Based on this analysis, the final model uses only `age`. We use `DecisionTreeClassifier` with a single categorical feature, the tree simply learns a lookup table that maps each age group to its most common income level in the training data. The data is split into 80% training data and 20% test data, stratified by income so that both sets have the same share of each level. The code is in `src/income_model.py`. The first attempt with all fields is kept in the Git history under the tag `model2-all-features`(https://github.com/ruoxicao77i/cs528-hw6/blob/model2-all-features/src/income_model.py).

Accuracy limit. Table 2 also shows why the accuracy cannot be much higher. Knowing the age reduces the choice from eight income levels to three, which raises the accuracy from 12.5% to about 33%. But the three remaining levels are equally likely, so no model can tell them apart. So about one third is the expected upper bound for any model on this data, since none of the other fields add information. Results slightly above 33.3% come only from random variation in the data and in the train/test split.

5 Results

Model	Features	Train	Test (mean)	Test range
Decision tree	all features	76.8%	33.5%	32.8–33.9%
Decision tree	age only	33.8%	33.6%	33.3–33.9%
Decision tree	age + gender	34.1%	33.6%	33.2–34.4%
Decision tree	age + country	40.2%	33.4%	32.7–33.6%
Decision tree	age + hour	35.7%	33.5%	33.1–34.0%
Random forest	all features	76.8%	33.6%	33.4–33.7%

Table 3: Income model accuracy with different features (10 seeds for decision trees, 5 for the random forest)

The 35% target. The test accuracy depends on the random seed, but only slightly. Over 10 seeds, our final model ranges from 33.1% to 33.8%, and none of the runs reach 35%. Across all the feature sets we tried, the best single run was 34.4%. This matches the analysis in Section 4.2: the theoretical upper bound of accuracy with this data is one-third, so 35% can only be reached by chance, when the test set happens to favor the guessed levels.